

STRAIKER • THE AGENTIC SECURITY COMPANY

STAR LABS THREAT REPORT • VOLUME I • JULY 14, 2026

THE YEAR AGENTS ENTERED THE **WORKFORCE**

AiPT, LAVA, and the Straiker STAR Framework for agentic security. Thousands of real-world exploits run against coding, productivity, and first-party AI agents, and the model defenders need to act on what we found.

Defenders prepared for hostile users and rogue models.
Here the model obeys, the user is innocent and the context
is the weapon.

– STRAIKER STAR LABS

CONTENTS

01 · Foreword	3
02 · Executive Summary	4
03 · The STAR Framework	5
04 · Coding Agents	10
05 · Productivity Agents	17
06 · First-Party Agents	22
07 · The Agentic Supply Chain	25
08 · Defenders	28
09 · Methodology and Sources	30

AGENTS ARE THE NEW ATTACK SURFACE

AI making decisions for us was science fiction. Two years ago, enterprises were simply blocking genAI sites like ChatGPT. Now agentic AI adoption is integral to corporate strategy, the difference between being a leader and being left behind. The productivity wave arrived first, with agents reading inboxes, browsing the web, and generating images. Enterprises are also building their own agents on Microsoft Foundry, Amazon Bedrock AgentCore, Google Gemini Enterprise, Databricks Mosaic AI, and Snowflake Cortex. Now coding agents are becoming part of the enterprise operating system: Cursor, Claude Code, and GitHub Copilot live in every engineer's IDE and ship code straight into production. Each phase has moved faster than the last, and each has brought an attack surface the previous controls were never built to see. The pattern is consistent: the more capable and connected the agent, the larger the attack surface it hands the enterprise.

Adversaries are riding the same wave with agentic tools of their own. STAR Labs has documented a new class of attacker we call **AiPT**: AI-powered persistent threats. AiPTs operate with agentic offensive toolkits such as Cyberspike Villager. Their reconnaissance runs automatically, their exploits are generated for the specific target, and their persistence sits in the semantic layer traditional defenses cannot read. Agents are building agents.

AI needs AI to secure it. CVE scanners and SAST read code, EDR reads endpoints, and firewalls read packets. None of them read context, and context is the surface that decides whether an agent ships a fix or wipes a Drive. Securing an agent requires watching what it watches, at the speed it moves, across the four architectural layers it lives in: application, model, tools and MCP, and data.

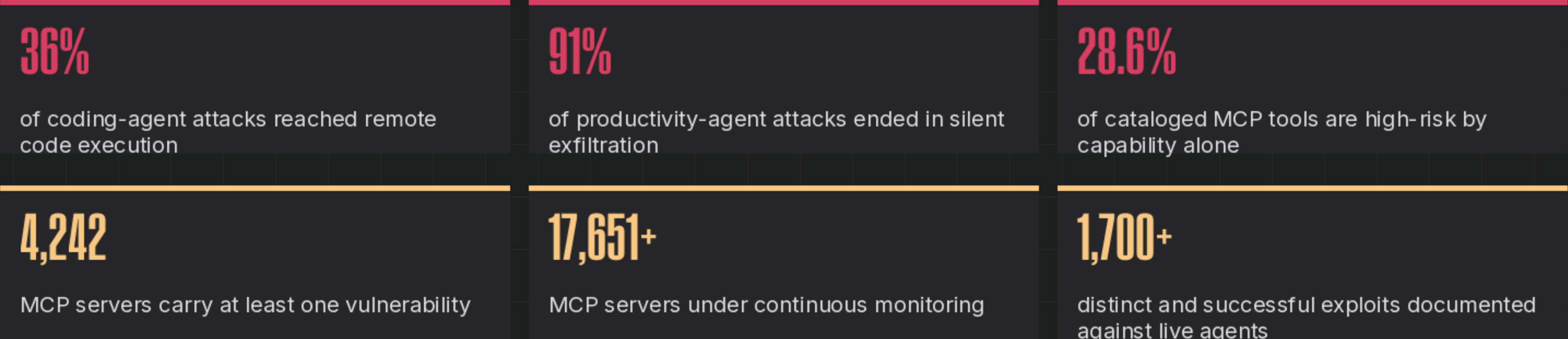
The **Straiker STAR Framework for AI Agent Security** (STAR = Straiker AI Security Research) organizes the agentic attack surface across those four layers and the three agent types where compromise has the highest enterprise impact: first-party, productivity, and coding agents. The sections that follow turn that surface into a model defenders can act on.

Vinay Pidathala

VP OF AI SECURITY RESEARCH • STRAIKER STAR LABS

THE YEAR AGENTS ENTERED THE WORKFORCE

Agentic AI is becoming the operating system of the business, and AI agents are its fastest-growing workforce. We ran adversarial prompts across many different agent form factors — coding, productivity, and custom — and the results were astonishing. Out of all the successful attacks, here's what we found:



SIX FINDINGS

- Coding agents are the highest-risk AI deployment.** 36% of successful attacks reached remote code execution, defeating sandboxes via dotfile poisoning, README injection, and parser differentials.
- Productivity agents fail silently, at scale.** 91% of successful attacks ended in silent data exfiltration, with no jailbreak, phishing link, or malware. The agent reads attacker-crafted content the same way it reads anything else, and acts on it.
- MCP is the shared, ungoverned supply chain.** Of 17,651+ tracked servers, 4,242 carry a vulnerability; 28.6% of 130,667 cataloged tools are high-risk by capability alone, able to execute code or reach the file system, database, or network. This is the npm crisis with autonomous agents holding the keys.
- Custom agents inherit enterprise-wide blast radius.** Agents on Foundry, Bedrock, Gemini, and Databricks run with internal-system trust; one compromise propagates in seconds.
- A new adversary class: AiPT.** AI-Powered Persistent Threats run agentic offensive toolkits like Cyberspike Villager, with reconnaissance, exploits, and persistence at agent speed.
- A new vulnerability class: LAVA.** Language-Augmented Vulnerabilities in Applications are what AiPTs exploit, and what the Straiker STAR Framework finds and contains.

A MAP OF WHERE ATTACKS HAPPEN

WHY AGENTIC SECURITY NEEDS A NEW FRAMEWORK

Several frameworks already address pieces of this problem. OWASP led the field, first with the Top 10 for LLM Applications and then with the Top 10 for Agentic Applications. Google's Secure AI Framework decomposes AI systems into four functional components. MITRE ATLAS organizes adversary tactics in the ATT&CK pattern. The NIST AI Risk Management Framework approaches risk through governance functions. Each is valuable. None maps an agent's architecture to its deployment context, where attacks actually happen.

The **Straiker STAR Framework** answers that on two axes at once. The first axis is **agent architecture**: every AI agent, regardless of vendor, runs on the same four layers (application, model, tools and MCP, and data), and attacks enter at one layer and chain across the others. The second axis is **deployment context**: enterprise agents land in one of three roles, each with a distinct trust posture and blast radius. Their intersection is where attacks happen, and each cell is populated by what STAR Labs has actually observed AiPTs do against real agents.

	CODING AGENTS ENDPOINT • LOCAL SHELL	PRODUCTIVITY AGENTS STRADDLE • OAUTH SCOPE	FIRST-PARTY AGENTS REMOTE • OPERATOR TRUST
APPLICATION  Orchestration, state, approval gates	NO GATE Agent-generated shell commands auto-execute	OVER-SCOPE OAuth over-scope beyond the task	NO GATE Outputs act on internal APIs with operator trust
MODEL  Reasoning & instruction hierarchy	INJECTION Poisoned code / README reframes intent	INJECTION Indirect injection from email & documents	INJECTION Operator-trust instruction override
TOOLS & MCP  Connectors, servers, APIs	SUPPLY CHAIN Local MCP binary poisoning, rug pulls	EXCESSIVE AGENCY Excessive agency across connected tools	REGISTRY Weak per-tool auth, rogue registration
ONE MALICIOUS MCP SERVER HITS ALL THREE AT ONCE • 4,242 SERVERS CARRY ≥1 VULNERABILITY			
DATA  RAG, memory, docs, uploads	POISONING Poisoned README / dependency → laptop RCE	EXFILTRATION Gmail & Drive files exfiltrated to attacker's email	POISONING Poisoned RAG / connector → enterprise breach
DOMINANT OUTCOME	RCE on the developer endpoint	91% end in silent data exfiltration	Enterprise-wide blast radius

The STAR Framework: four architectural layers across three agent types. Tools & MCP is the surface shared by all three.

AIPT AND LAVA: NAMING WHAT'S NEW

Defenders need vocabulary for what they are defending against. **AiPT** (AI-Powered Persistent Threats) are adversaries operating with agentic offensive toolkits; Cyberspike Villager, an AI-powered Cobalt Strike lookalike, is the canonical example STAR Labs has documented in the wild. Four properties set AiPTs apart from previous APT generations:

- **Automated reconnaissance.** Target enumeration, surface mapping, and intelligence gathering happen at agent speed.
- **Contextual exploit generation.** Payloads are crafted to the specific context of the target agent: the email it reads, the tool it trusts, the README it processes.
- **Semantic persistence.** Persistence operates in the layer of meaning traditional defenses cannot read. There is no implant to detect, because the implant is the agent doing exactly what it was instructed to do.
- **Agent-native operation.** AiPTs do not bolt AI onto existing tradecraft. They are agents themselves, hunting and exploiting other agents.

A live example of AiPT economics is the fake Claude Code campaign STAR Labs documented in 2026 (Section 4): automated impersonation at the scale of 88 domains, paid ad placement to out-rank the real tool, and a malware family rebuilt specifically to steal AI-coding-agent credentials. The target is no longer just the agent's behavior; it is the trust developers place in the tools the agent ships inside.

LAVA (Language-Augmented Vulnerabilities in Applications) is the vulnerability class AiPTs exploit, a new threat class STAR Labs identified at the intersection of traditional application vulnerabilities and AI-driven language capabilities. A classic AppSec vulnerability is an unintended hole in the code path; a LAVA is an exploit in the **language path** the agent reasons over, where the payload is content the agent reads and follows. Three examples make it concrete:

- An agent fetches instructions from a README or website, then accesses an internal URL, causing **SSRF**.
- An MCP tool connected to an agent is vulnerable to **SQL injection**, and the agent passes attacker-shaped input straight through to it.
- An agent web UI carries a **cross-site scripting (XSS)** flaw an attacker exploits to exfiltrate data.

The underlying flaw may show up in a scan, but what no CVE scanner, SAST tool, or endpoint agent can see is the agent deciding, in language, to deliver the payload. That decision is detectable only by an agent watching what other agents are reading and deciding.

THE FOUR ARCHITECTURAL LAYERS

Every AI agent runs on the same four layers, and attacks enter at one and chain across the others. Knowing which layer is exercised tells defenders which control catches the attack.

- **Application layer.** The interface, orchestrator, and logic that turn model outputs into actions, controlling state and approval gates. Missing approval gates escalate any model-layer compromise into real-world impact: an agent that decides to wipe a Drive will wipe a Drive if no gate stops it.
- **Model layer.** The LLM reasoning engine, where instruction hierarchy can be confused or overridden through prompt injection (direct or indirect), jailbreak, and instruction-hierarchy attacks. STAR Labs found that **75% of tested agentic applications are vulnerable to injection at this layer.**
- **Tools and MCP layer.** The tools, MCP servers, connectors, and APIs the agent calls. This layer expands what an agent can do, which expands the blast radius of any compromise. Threats include tool poisoning via descriptions, output injection via tool results, rug pulls via auto-update, tool-name spoofing, unauthorized tool registration, and supply-chain attacks on MCP binaries. Section 7 covers the MCP supply chain in depth.
- **Data layer.** RAG corpora, persistent memory, connector-sourced documents, and user uploads: all untrusted input ultimately feeds the model from here. Threats include RAG poisoning, cross-session memory poisoning, context-window stuffing, and malicious document upload. Most LAVAs land here first, even when they exploit a different layer downstream.

4	APPLICATION INTERFACE • ORCHESTRATION • APPROVAL GATES MISSING APPROVAL GATES • UNCHECKED OUTPUT ACTION EXECUTES SURFACES AS ACTION	BREAK IT Approval gates on every irreversible action
3	TOOLS & MCP TOOLS • MCP SERVERS • CONNECTORS • APIS TOOL POISONING • TOOL-RESULT INJECTION • RUG PULLS • ROGUE REGISTRATION • BINARY SUPPLY CHAIN DECISION EMITS TOOL CALLS	BREAK IT Sign & pin servers; per-tool authorization
2	MODEL LLM REASONING • INSTRUCTION HIERARCHY PROMPT INJECTION (DIRECT / INDIRECT) • JAILBREAK • INSTRUCTION-HIERARCHY ATTACKS • 75% OF AGENTIC APPS INJECTABLE HERE UNTRUSTED INPUT FEEDS THE MODEL	BREAK IT Instruction-hierarchy enforcement & injection guardrails
1	DATA RAG • MEMORY • DOCUMENTS • UPLOADS RAG POISONING • MEMORY POISONING • CONTEXT STUFFING • MALICIOUS UPLOAD • ATTACK ENTERS HERE	BREAK IT Treat all input as untrusted; check provenance

One attack, four layers: a single compromise enters at Data and escalates to real-world impact, with the control that breaks each step.

Attacks rarely live in a single layer. A canonical compromise: an attacker plants instructions in data-layer content (a poisoned README, a malicious calendar invite); the model layer reads it as operator-level instruction; the tools and MCP layer executes the resulting tool calls; and the application layer surfaces the outcome with no approval gate in the way.

THE THREE AGENT TYPES IN ENTERPRISE SCOPE

Enterprise agents land in one of three deployment roles, each with a distinct trust posture and blast radius. The same layer means a different thing in each: data-layer poisoning of a coding agent ends in code execution on a developer's laptop; of a productivity agent, exfiltrated PII from Drive; of a first-party agent, enterprise-wide compromise.

A second axis sits beneath deployment role: **where the agent runs**. Endpoint agents run on the user's own machine with direct access to the local filesystem, shell, and credentials, so a compromise reaches the user's full local privilege. Remote agents run on managed infrastructure reached over the network, bounded by the agent's service identity but propagating across whatever internal systems that identity can reach. Coding agents are mostly endpoint; first-party agents mostly remote; productivity agents straddle both.

CODING AGENTS ENDPOINT • LOCAL SHELL CURSOR • CLAUDE CODE • GITHUB COPILOT • CHATGPT CODEX TRUST POSTURE Direct access to local filesystem, shell, and package managers. PRIMARY ATTACK SURFACE RCE via shell-init modification and workspace-boundary violations; local MCP binary supply chain. DOMINANT OUTCOME Remote code execution on the endpoint. DEFENDER'S JOB Sandbox; sign & pin MCP binaries; disable auto-execution.	PRODUCTIVITY AGENTS STRADDLE • OAUTH SCOPE CLAUDE COWORK • CHATGPT ENTERPRISE • SLACK AI • ASANA AI • BROWSER AGENTS TRUST POSTURE OAuth-scoped access to connected SaaS; processes untrusted external content by design. PRIMARY ATTACK SURFACE The connectors that make them useful are both the injection and the exfiltration channel. DOMINANT OUTCOME 91% end in silent data exfiltration. DEFENDER'S JOB Minimize connector scopes; approve irreversible actions; treat tool results as untrusted.	FIRST-PARTY AGENTS REMOTE • OPERATOR TRUST MICROSOFT FOUNDRY • BEDROCK AGENTCORE • GEMINI ENTERPRISE • DATABRICKS • SNOWFLAKE TRUST POSTURE Elevated operator trust; broad access to internal APIs, RAG, and enterprise data fabrics. PRIMARY ATTACK SURFACE Tool registry and per-tool authorization; internal content trusted by default. DOMINANT OUTCOME Enterprise-wide blast radius in seconds. DEFENDER'S JOB Harden the tool registry; per-tool authorization; treat internal content as untrusted.
--	---	---

The three deployment roles, by trust posture, primary attack surface, and dominant outcome.

- **Coding agents.** Direct access to local filesystems, shell execution, and package managers (Cursor, Claude Code, GitHub Copilot, ChatGPT Codex, Snowflake Cortex Code). Their risk is dominated by remote code execution via shell-init modification and workspace-boundary violations, plus binary supply-chain risk from local MCP servers that no remote deployment shares. Defender's job: sandbox, sign and pin MCP binaries, and disable auto-execution of agent-generated shell commands.
- **Productivity agents.** Read and act on connector-sourced content (email, Drive, Calendar, GitHub, Slack) via OAuth and MCP, processing high volumes of untrusted external content by design. STAR Labs sees a near-monoculture risk profile: **91% of successful attacks result in silent data exfiltration**. The connectors that make them useful are also the injection and exfiltration channels. Defender's job: minimize connector scopes, require approvals for irreversible actions, and treat tool results as untrusted.
- **First-party agents.** Built and deployed by the organization on platforms like Microsoft Foundry, Amazon Bedrock AgentCore, Google Gemini Enterprise, Snowflake Cortex, and Databricks Mosaic AI. They operate with elevated operator trust and broad access to internal APIs, RAG corpora, and data fabrics, so a compromise has an enterprise-wide blast radius. Defender's job: harden the tool registry, enforce per-tool authorization, and treat internal connector content with the same suspicion as any third-party input.

THE AGENTIC SUPPLY CHAIN: THE SHARED SURFACE

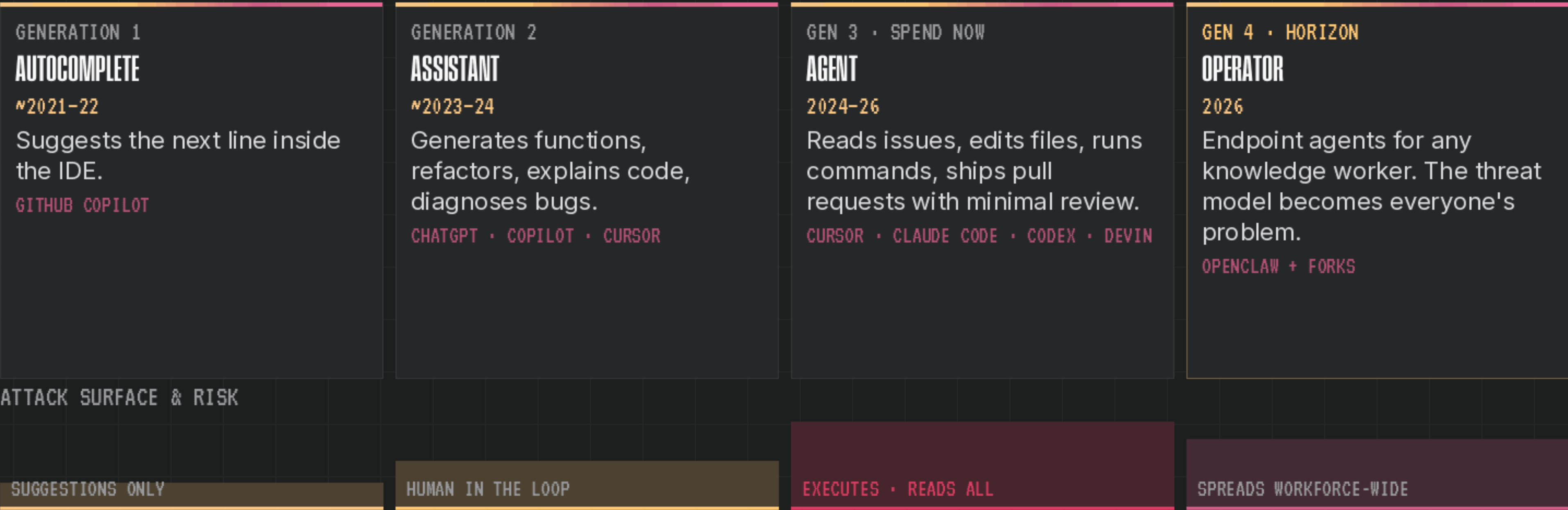
Agents reach the systems their human counterparts use through many paths: direct APIs, native function calling, Skills, plugins, and OAuth-mediated connectors. Together these form the agentic supply chain, the shared surface every agent type depends on. Of these, Model Context Protocol has emerged as the highest-leverage shared attack surface STAR Labs observes in the wild. MCP standardizes how any agent connects to any tool, which drove rapid enterprise adoption, and the same standardization is why a single malicious MCP server can compromise a first-party agent, a productivity agent, and a coding agent simultaneously.

MCP is not the only shared supply chain. **Skills**, the packaged instruction files an agent loads as trusted context (popularized by Claude's Agent Skills and Vercel's open Skills ecosystem), are the second: a poisoned Skill is indirect prompt injection with a distribution channel, and one bad MCP server or Skill reaches every agent type at once. Section 7 quantifies both.

THE HIGHEST-RISK DEPLOYMENT

WHY CODING AGENTS ARE THE HIGHEST-RISK DEPLOYMENT

Coding agents have moved through three generations in three years. The first was autocomplete, embedded in the IDE and helpful within a single line of code. The second was the assistant, which could generate functions, refactor blocks, explain unfamiliar code, and diagnose bugs on request. The third is the agent: a system that reads issues, plans changes, edits files, runs commands, installs packages, writes and runs tests, fixes bugs across the codebase, and ships pull requests, often with minimal human review.



Four generations, one rising risk curve. Generation 3 is where enterprises spend now; the Operator generation is on the horizon.

The third generation is where enterprises spend today: Cursor, Claude Code, GitHub Copilot, ChatGPT Codex, Cognition Labs' Devin, Replit Agent, Vercel v0, and enterprise-data-native variants like Snowflake Cortex Code. A fourth is on the horizon: truly autonomous agents teams are piloting now but have not yet deployed in production. **OpenClaw**, with peers like Manus and AutoGPT, is its leading edge. Whether it is an evolution of the Generation 3 agent or a revolution beyond it, it inherits the same endpoint posture (local execution, shell access, a trusted-reader relationship to the whole machine) and the same threat model, but its center of gravity is no longer code. Built for any knowledge worker who needs multi-step reasoning, OpenClaw-class agents push the endpoint attack surface out of the IDE and into the general workforce. As they spread beyond engineering, the coding-agent threat model here becomes everyone's problem.

That makes coding agents the highest-risk AI deployment in any enterprise today, and the data confirms it: **more than a third of successful attempts against coding agents reached remote code execution**, a higher rate than productivity agents (where the dominant outcome is data exfiltration), higher than first-party agents (internal data movement), and orders of magnitude above traditional CVE-driven RCE. Three properties make coding agents uniquely exposed:

- **Direct execution capability.** Coding agents run shell commands, modify files, and install packages by design. The "agent decides, then executes" loop is the product; no safe interpretation of agent output avoids the execution step.
- **Trusted reader posture.** They read every artifact a developer would: READMEs, code comments, dependency docs, config files, shell-init scripts, test fixtures, and bug reports. Each one is a LAVA delivery channel.
- **Local privilege footprint.** They live on developer workstations with SSH keys, cloud credentials, `.env` files, and source. A compromise reaches not just the current session but the developer's entire identity.

THE THREE OUTCOMES THAT DEFINE CODING-AGENT RISK

STAR Labs sorts successful coding-agent attacks into three outcomes, in descending order of how often they end an engagement.

- **Remote code execution.** Attacker-controlled code runs inside the agent's own runtime. **36% of all successful attempts against coding agents reached RCE**, a higher rate than any other agent type and orders of magnitude above traditional CVE-driven RCE.
- **Data exfiltration.** Source code and secrets leak through the agent's own tool calls. A compromised agent reads `.env` files, cloud credentials, and proprietary source, then ships them out with the same network primitives it uses for legitimate work. To endpoint detection it looks like an authenticated developer using a normal tool.
- **Sandbox escape.** The agent breaks out of its execution sandbox to reach the host. STAR Labs disclosed a sandbox-escape finding to a major enterprise data-platform vendor whose coding agent was in evaluation. Sandbox boundaries fall to dotfile poisoning, shell-parser differentials, and validator-chain gaps.

These properties and outcomes combine into the attack chain STAR Labs documents most often. It is examined next, followed by two case studies: the Cursor sandbox escape STAR Labs disclosed in April 2026, and a live infostealer campaign impersonating Claude Code itself.

THE CANONICAL CODING-AGENT ATTACK CHAIN

STAR Labs sees the same five-step chain in most coding-agent compromises. Knowing the chain is the prerequisite to breaking it, because each step has a different control.

STEP 1 • DATA ENTRY Attacker plants instructions in content the agent reads. PAYLOAD README • .cursorrules • MCP tool desc BREAK IT Treat all repo content as untrusted.	STEP 2 • MODEL INTERPRET Treated as operator instruction, not data. This is LAVA. PAYLOAD data parsed as instruction BREAK IT Runtime instruction-hierarchy enforcement.	STEP 3 • TOOL EXECUTE Issues shell commands, writes files, installs packages. PAYLOAD curl <host> sh BREAK IT Disable auto-exec; sandbox the shell.	STEP 4 • PERSIST ESCAPE Writes a shell-init file or escapes the sandbox parser. PAYLOAD >> ~/.zshenv BREAK IT Block shell-init writes; pin binaries.	STEP 5 • APP OUTCOME Remote shell, stolen credentials, backdoored pull request. PAYLOAD reverse shell • stolen .ssh/.env BREAK IT Scope credentials; monitor identity.
--	--	---	--	--

SEEN IN THE WILD • NOMSHUB / CURSOR SANDBOX ESCAPE • APRIL 2026

A README injection (1) was read as instruction (2), ran shell commands that bypassed the sandbox parser (3) and wrote to ~/.zshenv (4), opening a persistent remote shell (5). On the endpoints STAR Labs monitored, no endpoint-detection alerts fired.

From a poisoned README to full compromise, and the control that breaks each step. Seen in the Nomshub / Cursor sandbox escape.

- **Step 1 - Data-layer entry.** The attacker plants instructions in content the agent will read: a README in a cloned repo, a docstring in a dependency, a code comment, a config file like `.cursorrules`, a package's documentation, or an MCP tool description. The instructions are natural language, sometimes disguised as documentation.
- **Step 2 - Model-layer interpretation.** The agent reads the poisoned content and treats it as operator instruction rather than untrusted data. This is the instruction-hierarchy failure that defines LAVA. Once the model accepts the content as instruction, the rest of the chain follows from the agent doing its job correctly.

WHY MODEL GUARDRAILS AREN'T RUNTIME SECURITY

Every frontier lab ships safety training and built-in guardrails. Those guardrails matter as a baseline, but they are commodities now: trained for general safety, bypassed regularly, and blind to the specific runtime context of an enterprise agent. Runtime security catches what guardrails miss.

- **Step 3 - Tool-layer execution.** The model issues shell commands, writes files, or installs packages to satisfy the injected instructions. Coding agents do this by design. There is no anomaly to detect: the agent is calling its tools exactly the way it was supposed to.
- **Step 4 - Persistence or escape.** The payload writes to a shell-init file (`~/.zshenv`, `~/.bash_profile`, `~/.bashrc`), drops a malicious binary, modifies the agent's own config, or exploits a sandbox-parser differential to break out. Persistence means re-execution every time the developer opens a new shell.
- **Step 5 - Application-layer outcome.** Remote shell tunnel, exfiltrated credentials, modified source, or backdoors planted in a pull request. The outcome is realized on the developer's machine and immediately reaches the developer's identity surface: SSH keys, cloud credentials, and signed commits.

The chain is concrete. STAR Labs' **Nomshub** disclosure in April 2026 documented a critical sandbox-escape vulnerability in Cursor that walked through every step: a README prompt injection (Step 1) treated as instruction (Step 2) executed shell commands (Step 3) that bypassed Cursor's sandbox parser via shell builtins and wrote to ~/.zshenv (Step 4), activating a persistent remote shell tunnel that survived restarts (Step 5). Endpoint detection did not fire.

CASE STUDY: THE FAKE CLAUDE CODE CAMPAIGN AND AI-CREDENTIAL THEFT

The coding-agent threat is not only what a poisoned repository can make an agent do; it is also what an attacker can steal by impersonating the agent itself. STAR Labs tracked a live infostealer campaign that cloned Claude Code, JetBrains, NotebookLM, Cline, Comet, and Snowflake install pages, then bought Google Ads to rank a fake Claude Code page above the real one.

88	10+	6
cloned domains	hosting platforms	impersonated products

Products cloned: Claude Code , JetBrains , NotebookLM , Cline , Comet , Snowflake

Hosted across: Squarespace , GitHub Pages , Cloudflare Workers & Pages , Netlify , Tencent EdgeOne

The lure exploits how AI tools are adopted. A developer copies a shell command from a pixel-perfect clone of the docs and runs it with full system privileges. There is no installer to scan and no binary to inspect.

```
THE LURE - CURL SEPARATOR
curl -fsSL https://claude.ai/install.sh ; [ hidden payload . full system privileges ]
```

The visible curl to claude.ai is a decoy. A shell separator runs the real payload in the foreground; no installer, no binary to scan.

What the command pulls down is, for the moment, conventional: today's payloads are commodity infostealers and ransomware, not yet bespoke, agent-aware malware. STAR Labs treats that as a timing gap, not a ceiling, and expects purpose-built payloads as AI-agent credentials climb the target list. What has already changed is the targeting.

What the malware steals is the part that matters here. This is the first infostealer STAR Labs has documented built to harvest credentials from AI coding assistants specifically.

WHAT IT STEALS	HOW IT HIDES . ACRSTEALER	COMMAND AND CONTROL
Cline API keys and provider credentials, Continue.dev config, and Snowflake SSH session tokens, alongside the usual browser and crypto-wallet theft.	Runs entirely in memory, resolves Windows APIs by hash to dodge EDR hooks, and opens raw sockets via the kernel AFD driver so WinINet and WinHTTP monitoring sees nothing.	Runs through a Binance Smart Chain smart contract: no domain to seize, and no server to shut off.

THE TAKEAWAY FOR DEFENDERS

The credentials your developers use to authenticate AI coding agents are now a primary target, ranked alongside crypto wallets and browser sessions. An agent's API keys and session tokens deserve the same protection as any other production secret.

OTHER HIGH-SEVERITY PATTERNS

Beyond the canonical chain, six patterns recur often enough across coding-agent engagements to flag.

Malicious packages Agents trust the name in the manifest. A typosquat (<code>numpy</code> vs <code>numppy</code>) lands attacker code the moment the agent reads the install instruction.	Untrusted rules files <code>.cursorrules</code> and <code>.claude/CLAUDE.md</code> load as system-level instructions. A cloned repo can silently rewrite how the agent behaves for every session.	Network-primitive exfiltration With <code>curl</code> and <code>wget</code> in the tool set, a compromised agent ships <code>.aws/credentials</code> or <code>.env</code> out in one command that looks like normal dev activity.
Model-endpoint override Env vars can redirect the agent's LLM calls to an attacker endpoint, so every prompt, snippet, and credential it reads flows to the attacker.	RAG & codebase poisoning Attacker-controlled commits or comments in an indexed repo persist across users and sessions, steering future agent decisions unnoticed.	Destructive commands <code>rm -rf</code> or <code>DROP TABLE</code> runs if the agent is persuaded to. In the publicly reported PocketOS incident (April 2026), a Cursor agent wiped a production database and its backup in under nine seconds.

WHAT DEFENDERS NEED TO DO

Each step in the canonical chain has a different control. STAR Labs recommends the following baseline for any enterprise running coding agents in production.

	BREAK STEP 1: DATA-LAYER ENTRY Treat all content the agent reads as untrusted, including internal repositories. Sandbox unfamiliar repos on first clone. Apply hygiene to codebase RAG and persistent memory. Block ingestion of malicious config files from untrusted sources.
	BREAK STEP 2: MODEL-LAYER INTERPRETATION Runtime detection of LAVA patterns inside the context window: monitor what the model reads against what it acts on. This is where AI-on-AI runtime security applies, a second agent watching the first agent's context for injection the model itself cannot catch.
	BREAK STEP 3: TOOL-LAYER EXECUTION Allowlist permitted tools. Block network-egress tools (<code>`curl`</code>, <code>`wget`</code>) by default outside known endpoints. Require approval for shell commands above a risk threshold. Sign and pin MCP server binaries; disable auto-update in sensitive environments.
	BREAK STEP 4: PERSISTENCE OR ESCAPE Write-protect shell-init files. Run the agent inside a workspace boundary it cannot extend, and audit all writes outside it. Triage sandbox-escape attempts at the highest severity tier.
	BREAK STEP 5: APPLICATION-LAYER OUTCOME Require human approval for irreversible actions (database drops, mass deletions, force-pushes, external sharing). Monitor egress for exfiltration. Keep off-volume backups so destructive execution has a recovery path.

The cross-cutting control is **least privilege**: an agent granted broader access than the task needs expands the blast radius of every other failure. Minimizing permissions before deployment catches more attacks than detecting them after. Detection at runtime is where most of these controls live, because the same agent action looks legitimate or malicious depending on the context the agent was reading when it took it. That is the case for AI-on-AI runtime security, examined in Section 8.

THE QUIET FAILURE MODE

WHY PRODUCTIVITY AGENTS ARE THE QUIET FAILURE MODE

Productivity agents arrived earlier than coding agents and embedded themselves faster. They read your inbox, summarize your calendar, draft meeting notes, propose replies, browse the web on your behalf, and reach across SaaS connectors to pull the data they need. The wave spans chat assistants, workspace suites, browsers, and the long tail of vertical agents shipping inside every SaaS product enterprise teams already use.

CATEGORY	REPRESENTATIVE TOOLS AND PRODUCTS
 Chat assistants	ChatGPT, Claude, Gemini, Microsoft Copilot Chat
 Email and productivity suites	Microsoft 365 Copilot, Gemini for Workspace, Outlook and Gmail with copilots
 Calendar and scheduling	Outlook Calendar with Copilot, Google Calendar with Gemini, assistant-driven scheduling plugins
 Browser copilots and AI browsers	ChatGPT Atlas, Claude for Chrome, and other AI-native browsers and browser agents
 CRM and revenue agents	Salesforce Agentforce, CRM-native copilots such as HubSpot AI
 System-level computer agents	General-purpose digital workers acting across desktop apps and the web
 Embedded SaaS agents	Slack AI, Asana AI, Notion AI, Jira AI, plus a long tail of app-specific copilots

Every one of these agents shares a structural property: they process untrusted external content as a core function. Emails, calendar invites, web pages, and shared documents all arrive from outside the trust boundary, and the agent is built to read all of it and act on the user's behalf. That is the product.

It is also why productivity agents are the quiet failure mode. Where coding agents fail loudly (databases dropped, backdoored code, compromised machines), productivity agents fail silently. A productivity-agent compromise can ship a copy of your entire Drive to an attacker over an afternoon and never trip a single SOC alert.

THE 91% PATTERN: SILENT EXFILTRATION AS THE DOMINANT OUTCOME

91%

of successful productivity-agent attacks observed by STAR Labs ended in data exfiltration that needed no jailbreak, no human-clicked phishing link, and no malware on the endpoint.

The number is high enough to look like a typo. It reflects how productivity agents are built: they read and send by design. Three structural properties converge to make silent exfiltration the dominant outcome.

- **Access by design.** OAuth scopes give the agent broad standing read access across the SaaS estate. An attacker who hijacks the agent's intent inherits all of it.
- **Egress paths by design.** Agents send email, share documents, post messages, and write to Drive. Every legitimate action is also an outbound channel.
- **No instruction discrimination.** The LAVA mechanism in the productivity context: a user instruction and an attacker-embedded one look identical to the model.

The exfiltrated data has a recurring shape. The connectors most enterprises grant first and review least are also the ones STAR Labs sees in nearly every successful chain.

CONNECTOR	SHARE OF ATTACKS	ROLE IN THE ATTACK CHAIN
✉ Email (Gmail, Outlook)	68.8%	Primary injection delivery and exfiltration channel
📁 Shared drive (Google Drive, OneDrive)	56.2%	Data storage target for theft or destruction
📄 Docs and spreadsheets	14.1%	Secondary data exposure via shared documents
🔗 Code repositories	7.8%	Code and credential theft
📅 Calendar (Google, Outlook)	4.7%	Reconnaissance, meeting injection
💬 Team chat	3.1%	Internal communications exfiltration

THREE WAYS THE EXFILTRATION SHOWS UP

The 91% headline is one number with three faces, each seen across STAR Labs productivity-agent engagements.

- **Data exfiltration.** In one engagement, infrastructure secrets left through a PDF-library exploit: the agent parsed an attacker-crafted document, the library mishandled it, and credentials in the agent's context were carried out. The agent worked as designed and would not be flagged by traditional security monitoring tools.
- **Credential harvesting.** OAuth tokens and API keys stolen through tool misuse. A hijacked agent calls a connected tool in a way that surfaces the very tokens authorizing its access, handing the attacker standing entry to the SaaS estate with no password phished.

- **Excessive autonomy.** An agent asked to summarize an inbox proceeds to send, share, or delete, because nothing required it to stop and ask. The missing approval gate is the vulnerability.

BROWSER AGENTS DISTRIBUTE THE DAMAGE

Where connector-only agents show a 91% exfiltration monoculture, browser agents spread the harm across the live session. **39% of successful attacks on browser agents led to data exfiltration and 37% to direct browser harm** such as wipes, transactions, and deletions.

BROWSER-AGENT ATTACK OUTCOME	SHARE
Data exfiltration	39%
Direct browser harm (wipes, transactions, deletions)	37%
User data harvesting	14.8%
Credential and session abuse	7.4%
Arbitrary code execution	1.9%

These browser figures and the 91% describe two different agent populations, not one number split two ways: connector-only agents converge on silent exfiltration, while browser agents spread the damage between exfiltration and direct action. Both are productivity agents; the outcome shifts with how much live-session access the agent holds.

- **Browsing-history leak.** An indirect injection on a visited page silently harvests the user's full session history, mapping everywhere they have been and everything they have open.
- **Drive wipe via email.** In a STAR Labs demonstration against a leading AI browser, a single benign-looking email drove the browser agent to wipe a connected cloud drive, with no jailbreak and no user click.
- **Session hijacking.** OAuth tokens stolen through a cross-site browsing context let an attacker ride the authenticated session the user already established.

Browser agents operate against the live session, where credentials and authenticated state are reachable, so a single compromise can both steal data and take destructive action. The systemic insight: productivity agents are exfiltration tools by default. The only question is whether they exfiltrate to legitimate destinations, or to attackers as well.

This is not only a lab result. The live infostealer campaign STAR Labs documented in Section 4 already harvested credential stores from AI tools and browsers directly, so an AI browser's stored credentials and sessions are an explicit target for real-world stealers today, not a projected one.

THE CONNECTED-APP CASCADE

When a productivity agent is installed, the user grants it permission to read and act across their connected apps, usually as a single bundle approved once and forgotten. An agent installed to summarize an inbox each morning typically asks for full mailbox read access; one installed to draft replies asks for full read and write. The agent then operates with that access indefinitely, whether the task needs it or not. Once an attacker hijacks the agent's intent, every connected app it can touch becomes part of the same compromise.

1

Single agent compromise. An attacker-crafted email lands in the inbox; the agent reads it during a summarization run.

WHY IT WORKS

The agent treats inbox content as instruction-eligible.

2

Connected-app reach. The agent applies the injected instructions across every app it can touch: Drive, Calendar, Slack, GitHub.

WHY IT WORKS

Permissions were granted in a single bundle at install.

3

Data movement. Documents copied out via Drive sharing, internal information posted to an attacker Slack channel, calendar invites used for reconnaissance.

WHY IT WORKS

Each is a legitimate action the agent is authorized to perform.

4

Persistence. Mailbox rules created, recurring calendar events scheduled, automated workflows configured.

WHY IT WORKS

Productivity agents have access to the configuration surfaces that enable persistence.

The fix is two-part: narrow what the agent is allowed to touch, and watch what it actually does at runtime. Narrowing access stops the cascade from being possible; runtime monitoring catches it when narrowing is incomplete.

WHAT DEFENDERS NEED TO DO

Each step in the cascade has a different control. STAR Labs maps three controls to the cascade steps where they actually catch the attack.



BREAK STEP 1: COMPROMISE

Treat every inbound document, email, calendar invite, and shared file as untrusted, instruction-eligible content, including internal content from unfamiliar senders. This removes the LAVA delivery channel where 91% of attacks arrive.



BREAK STEP 2: CONNECTED-APP REACH

Narrow what the agent is permitted to touch. Grant only the access the task needs, and revoke unused permissions on a quarterly cadence. An agent without Drive permission cannot exfiltrate from Drive.



BREAK STEPS 3 AND 4: MOVEMENT AND PERSISTENCE

Require human approval for irreversible actions: external email, document sharing, mailbox rules, and connector-setting changes. A hijacked agent cannot complete the action without a person saying yes.

One control runs across all four steps: **runtime monitoring of the agent's context**. The model itself cannot tell a legitimate instruction from a poisoned one, so the runtime layer has to. It watches what the agent reads against what it does and flags the moment the two stop aligning with the user's intent. This is the agent-on-agent runtime security model the Striker STAR Framework introduces; Section 8 returns to it as the operational answer to LAVA across every agent type.

THE INTERNAL BLAST RADIUS

WHY FIRST-PARTY AGENTS CARRY THE LARGEST BLAST RADIUS

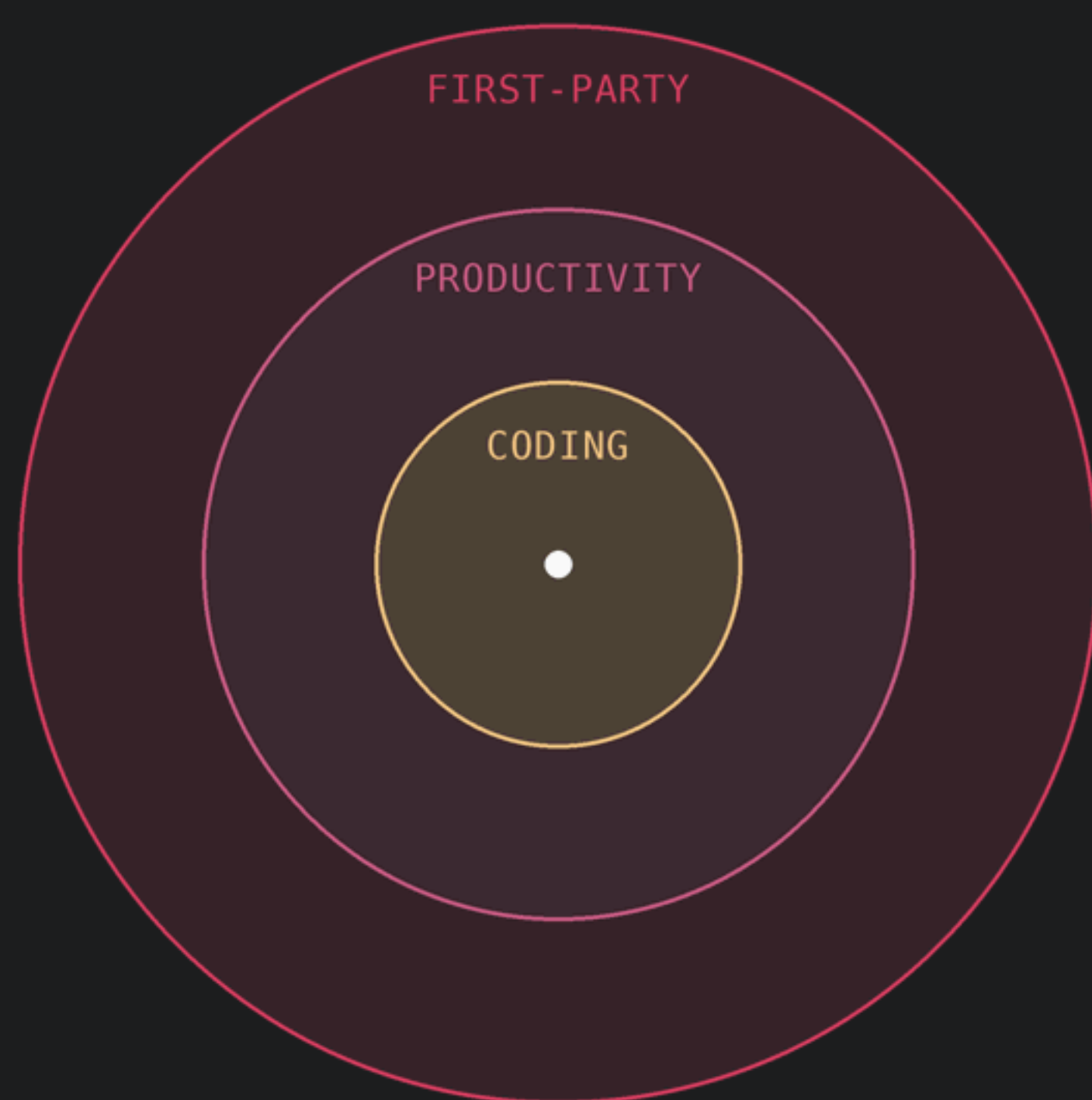
First-party agents are the ones an enterprise builds for its own use. They live on the enterprise's own infrastructure, connect to internal systems of record, and operate inside the trust boundary the organization has drawn around its data.

Every major cloud now ships a platform for building them, and each one deploys agents directly onto the enterprise's primary data store.

Microsoft Foundry Sits on Azure data services, the Microsoft 365 graph, and internal APIs.	Amazon Bedrock AgentCore Reaches S3, Redshift, and internal AWS services.	Google Gemini Enterprise Reaches BigQuery, Google Workspace, and internal APIs.
Snowflake Cortex Agents Operates directly over the Snowflake data cloud and its warehouses.	Databricks Mosaic AI Reaches lakehouse tables and Unity Catalog-governed data.	

The hyperscalers describe agents as enterprise infrastructure. Google frames them as "first-class distributed systems citizens, complete with identity, permissions, runtime environments, observability, and deployment pipelines." Snowflake calls its agent platform "the unified control plane for the agentic enterprise." Enterprises take the same view, deploying first-party agents with elevated trust and access to the data and systems the business runs on.

That elevated trust is also the structural risk. First-party agents operate inside the trust boundary, so a compromise reaches everything inside the boundary. Where productivity agents are constrained to the connected apps a user happens to have linked, and coding agents to the developer's workstation, a first-party agent compromise can reach the corporate data warehouse, the production database, the internal API surface, and the systems of record the business depends on.



■ CODING

The developer workstation and the production code it ships. 36% of successful attacks reach RCE.

■ PRODUCTIVITY

The connected SaaS estate. 91% of successful attacks end in silent exfiltration.

■ FIRST-PARTY

The enterprise data fabric. Enterprise-wide breach in seconds.

One compromise, three blast radii. The rings map reach, not likelihood or severity: coding agents are the highest-risk entry point (36% of successful attacks reach RCE), while first-party agents sit inside the enterprise boundary, so their reach is the whole business.

THE THREE WAYS FIRST-PARTY AGENTS FAIL

STAR Labs sees first-party agents fail in three recurring ways, each amplified by the fact that one agent serves the entire organization.



INTERNAL RAG POISONING

An attacker who can write to a source the agent indexes (a wiki page, a shared drive, a ticket) plants content that shapes what the agent later retrieves. Retrieval quality is the exposure: stale, wrong, or poisoned documents degrade a chatbot's answers and, in an agentic system, can steer its actions. Unlike a file a reviewer opens and reads, poisoned retrieval content acts silently at query time.

WHY IT'S ENTERPRISE-WIDE

The same agent serves every employee. One poisoned source steers every interaction with that agent.



OVER-PERMISSIONED TOOL REGISTRY

The agent is registered with access to every internal tool the team thought it might need. Attackers exploit the breadth, with no single tool needing to be vulnerable.

WHY IT'S ENTERPRISE-WIDE

A compromised agent can chain low-privilege tools into high-privilege outcomes that no individual tool would have allowed.



ENTERPRISE DATA FABRIC EXFILTRATION

The agent has standing access to data lakes, warehouses, and internal APIs to do its job. A LAVA-driven instruction redirects that same access to an attacker-controlled destination. The agent's job is to retrieve and combine internal data; the attacker only has to redirect the destination.

WHY IT'S ENTERPRISE-WIDE

The agent already holds the access and the task. The attacker changes only where the results are delivered.

The defender's job at this tier is preserving the elevated trust posture without inheriting the elevated risk. One builder shows what that discipline looks like in practice.



IN PRACTICE: EMERGENT HARDENED WINGMAN BEFORE LAUNCH

Emergent, an AI app-builder, had Straiker STAR Labs red-team its connector-enabled agent **Wingman** before launch. Ascend AI ran the patterns that define first-party risk, cross-connector pivoting, multi-hop trust laundering, and scheduled-task persistence, across 18+ integrations. The defenses held, and the team hardened further before launch, treating **model output as a proposal a separate control layer must clear**. Full story at straiker.ai/customers/emergent.

WHAT DEFENDERS NEED TO DO

What makes a first-party agent high-risk is less who built it than the context it is deployed into: the APIs, systems, and actions it can reach. An agent granted account-management powers holds them whether or not anything about it is technically vulnerable.

In June 2026, attackers talked Meta's AI support assistant into changing recovery emails on high-profile Instagram accounts, a dormant Obama-era White House page among them, then forced password resets to seize them. No exploit was used; the agent simply held the access and was socially engineered into using it, and Meta has since fixed it.

The defender's job is keeping that access from becoming the attacker's leverage. Three controls break the three failure modes.



BREAKS: INTERNAL RAG POISONING

Restrict RAG corpora to authenticated, authorized sources. Monitor what gets indexed for injection patterns. Apply source-provenance checks the way an AppSec team applies dependency-provenance checks.

WHY IT WORKS

Removes the LAVA delivery channel at the source. An agent that cannot index a poisoned wiki page cannot be steered by one.



BREAKS: OVER-PERMISSIONED TOOL REGISTRY

Apply least privilege to the tool registry the way enterprises apply least privilege to identity. Use per-tool authorization rather than blanket access. Audit every new tool registration.

WHY IT WORKS

Stops attackers from chaining low-privilege tools into high-privilege outcomes.



BREAKS: ENTERPRISE DATA FABRIC EXFILTRATION

Allowlist destinations for agent outputs. Require approval gates for cross-system data transfers. Monitor egress against the agent's declared task purpose.

WHY IT WORKS

Catches the redirect step. The agent retrieves the data either way; defenders ensure it only delivers where the business intended.

One cross-cutting control runs above the three: **runtime monitoring of context against action**. The same agent-on-agent runtime security model from Section 5 applies, with higher stakes because the blast radius extends across the enterprise. Section 8 returns to the full control set.

THE AGENTIC SUPPLY CHAIN

THE AGENTIC SUPPLY CHAIN TODAY

Agents reach external systems through several integration paths, and each one is a supply chain of its own with its own trust model and attack surface. Together they form the agentic supply chain: the servers, tools, skills, plugins, and connectors an agent pulls in to get work done.

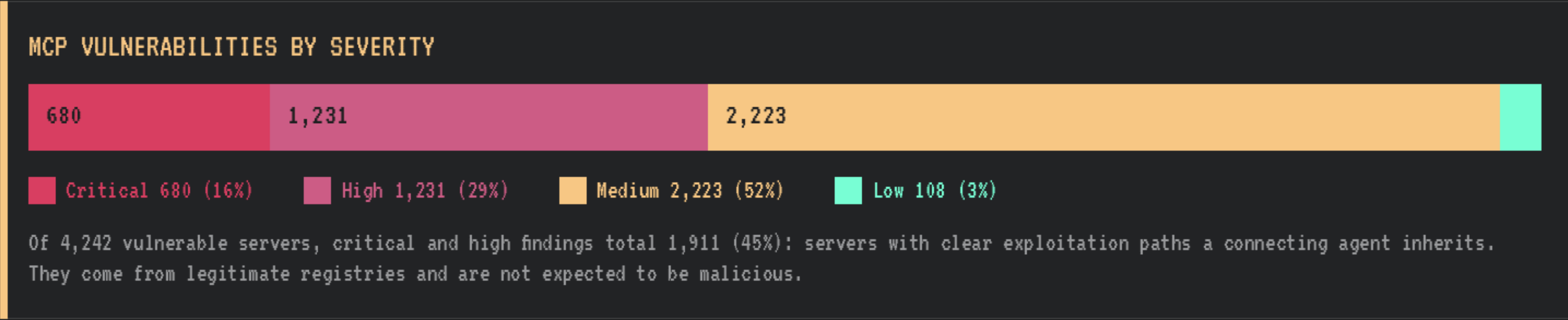
Beneath these agentic-specific paths sits the oldest supply chain of all: the open-source libraries the agent and its tools are built on. Assembled at vibe-coding speed, few have every transitive dependency reviewed line by line, so vulnerable and backdoored packages are absorbed upstream and inherited wholesale. Much real-world agent compromise still enters through these traditional software supply-chain weaknesses before any novel agentic path is touched.

INTEGRATION PATH	WHAT IT IS	NOTABLE EXAMPLES
Model Context Protocol (MCP)	Standardized protocol for tool registration and invocation across vendors.	Local and remote MCP servers across every major agent platform
Skills	Vendor-specific packaged capabilities loaded into the agent at runtime.	Claude Skills marketplaces
Plugins	Vendor-specific runtime extensions tied to a chat product.	ChatGPT plugins, Copilot extensions
Native function calling	Direct vendor SDKs that bind functions to an LLM's tool-use loop.	OpenAI function calling, Anthropic tool use, Google function calling
OAuth-mediated connectors	Authorization-scoped access to specific SaaS systems.	Productivity-agent connectors (Section 5)

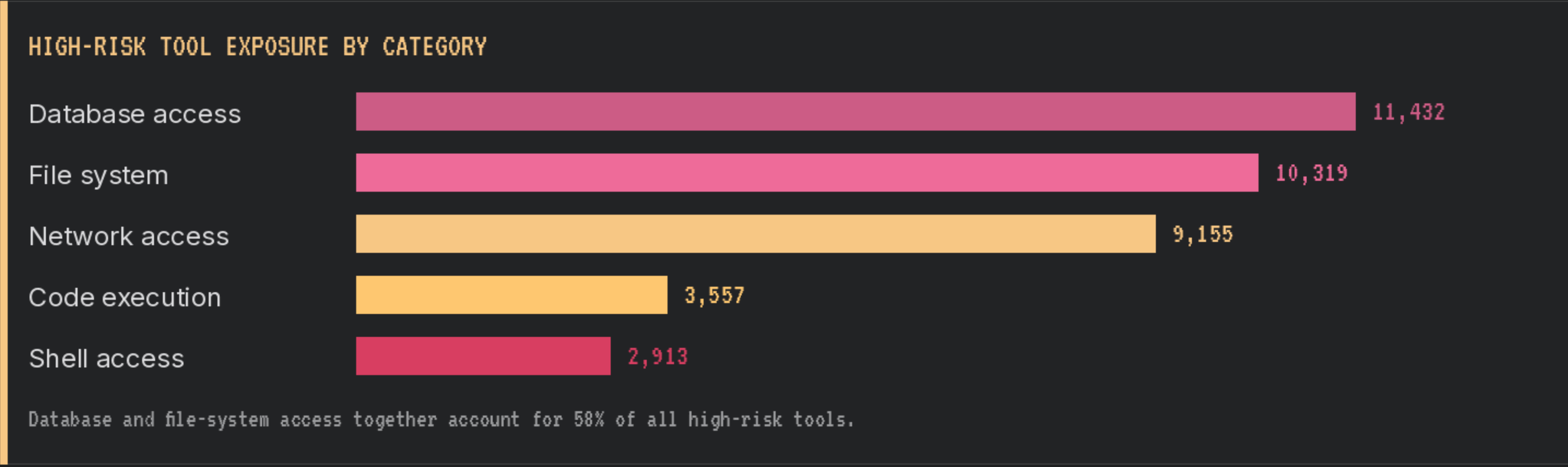
Of these, MCP carries the most concentrated risk: it standardizes across vendors, scales the fastest, and lacks the gatekeeping conventions that have grown up around the alternatives. Section 3 named it the highest-leverage surface; this section quantifies it, from a catalog STAR Labs maintains through continuous static, behavioral, and metadata analysis.

17,651+	4,242	28.6%
MCP servers under continuous tracking	servers carry at least one vulnerability	of cataloged tools are high-risk by capability alone

130,667	680	1,231
distinct tools catalogued, each an attack surface	critical-severity findings	high-severity findings



The shape of high-risk tool exposure is itself instructive.



An agent connected to an MCP server in either the database or file-system category inherits a credible path to structured-data theft or credential exfiltration in a single session.

The ecosystem's risk does not distribute evenly across runtime stacks.

RUNTIME STACK	CRITICAL	HIGH
Python	366	717
Node.js	246	417

Python carries more total critical findings, which tracks with data-heavy Python servers exposing direct database and file-system access. Node.js carries a heavy share of high-severity findings, reflecting the permissive nature of `npm` publishing and the absence of package-verification standards.

Three categories of MCP risk drive these numbers: **semantic risks** where the model reads all tool text as input and cannot reliably tell an instruction from a description, **local-server risks** where tampering and dotfile access escalate compromise to the host, and **remote-server risks** where missing authentication and network-boundary failures expose internal MCP infrastructure to external attackers.

CASE STUDY: CLAWHUB AND MOLTBOOK

STAR Labs documented an active agent-to-agent supply-chain attack across **ClawHub**, an agent extension marketplace, and **Moltbook**, an agent social network. Malicious skills published on ClawHub and promoted agent-to-agent on Moltbook drained Solana wallets through plaintext key storage and attacker-controlled payment aggregators.

THE TELL

Roughly **5% of analyzed ClawHub skills** were classified as overtly malicious or grey-area high-risk. Agent extension marketplaces inherit the same supply-chain risk as **npm** and **PyPI**, with one new wrinkle: an autonomous agent now holds the credentials.

The case sits at the tool-poisoning and semantic layers of the STAR Framework, and shows how MCP standardization extends the same supply-chain risk to every agent type that connects. Full mechanism and indicators of compromise are in the published research.

WHAT DEFENDERS NEED TO DO

MCP introduces a software supply chain where the dependencies are written in natural language and trusted as instruction. The defender's job is treating MCP servers like third-party code. Three controls break the three risk categories:

 SEMANTIC RISKS Treat tool descriptions, schemas, and outputs as untrusted input. Allowlist specific tools rather than auto-discover from registries. Validate tool metadata against an expected shape before the agent consumes it. WHY IT WORKS Removes the LAVA delivery channel inside tool metadata: instructions an attacker planted there never reach the model.	 LOCAL-SERVER RISKS Sign and pin MCP server artifacts. Disable auto-update in production. Sandbox local servers with restricted filesystem, network, and environment-variable access. WHY IT WORKS Stops tampering and rug-pulls. A pinned, signed server cannot silently change behavior after gaining trust.	 REMOTE-SERVER RISKS Require authentication on every MCP connection, including localhost. Validate the Host header to defeat DNS rebinding. Apply per-tool authorization, not blanket tokens. WHY IT WORKS Closes network-boundary failures. An external page that reaches an internal server still cannot invoke tools unauthenticated.
--	---	---

Above all three, run the open-source supply-chain playbook on MCP: software composition analysis, explicit version pinning, and review of every new server before install. Section 8 returns to the full control set.

WHAT THIS MEANS FOR DEFENDERS

ADOPT THE STAR FRAMEWORK

The value of the STAR Framework is operational: it shows defenders where attacks happen and which control breaks each step. Four cross-cutting controls run through every agent type in this report.

CONTROL	CODING	PRODUCTIVITY	FIRST-PARTY	MCP / SUPPLY CHAIN
Treat external content as untrusted instruction	READMEs, dependency docs, code comments	Inbound emails, calendar invites, shared docs	Internal RAG sources, wiki pages, tickets	Tool descriptions, schemas, server metadata
Narrow permissions to task scope	Tool registry, shell allowlists, filesystem boundaries	OAuth scopes per connector	Per-tool authorization, data-fabric access	Per-tool authorization, no blanket token access
Approve irreversible actions	Destructive shell commands, package installs	External email, document sharing, mailbox rules	Cross-system data transfers, output allowlisting	Tool invocations against sensitive systems
Monitor context against action at runtime	What the agent reads against what it executes	What the agent reads against what it sends	What the agent reads against what it touches	What the agent invokes against what tools should do

The STAR Framework as a control map: four cross-cutting controls across every agent type and the shared supply chain. Section 3 mapped where compromise happens; this maps what breaks it.

One control runs above the other four: **agent-on-agent runtime security**. The same agentic technology that introduces the LAVA attack class is what detects it at runtime. An agent watching another agent's context against its actions catches what the four static controls miss, including novel patterns the model itself cannot recognize. AI needs AI to secure it.

Three properties follow from an adopted framework:

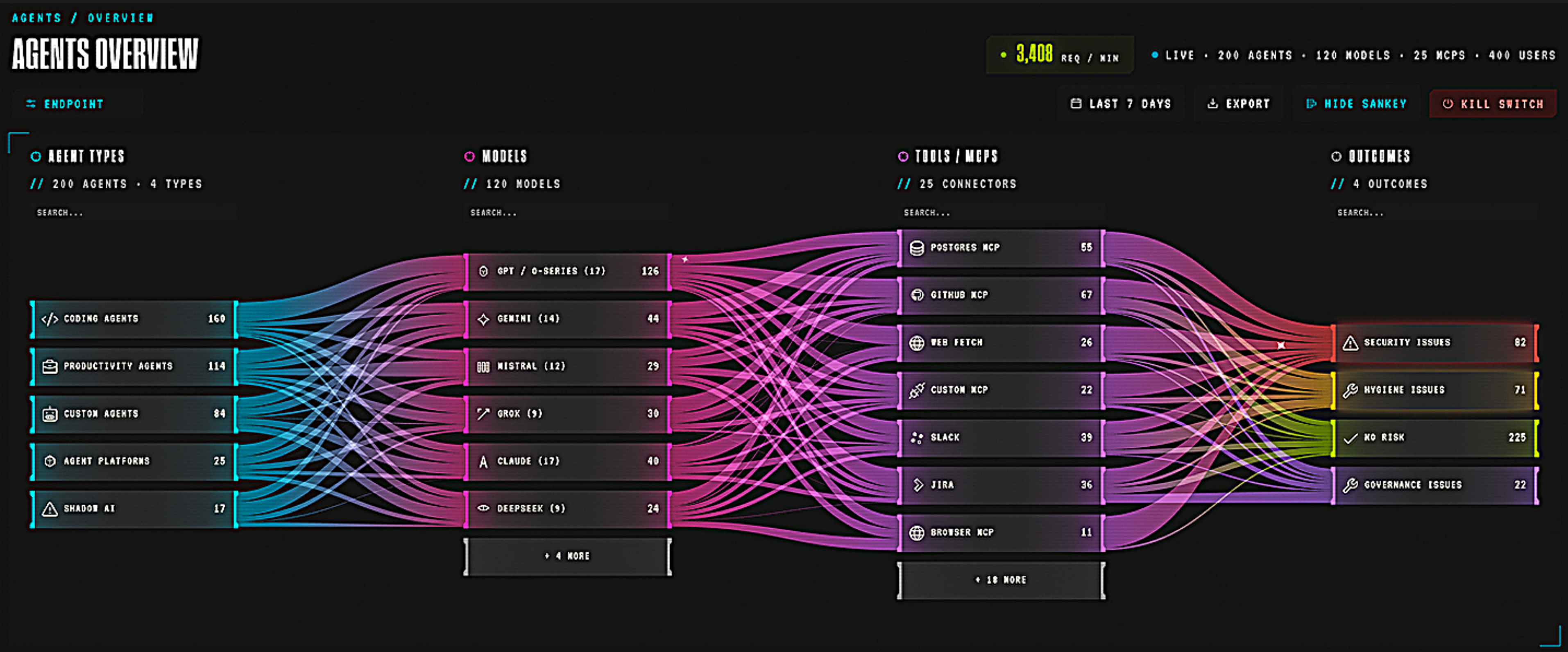
- **The attack surface stops being abstract.** Defenders can name which layer is being exercised, which agent type is exposed, and which control breaks the step.
- **Controls map to outcomes, not techniques.** A new injection pattern still lands at the data layer, pivots through the model layer, and tries to execute at the tools layer. The controls hold against the next technique without re-engineering.
- **The framework scales as the ecosystem scales.** New agent types and new extension paths inherit the same architectural treatment.

The full STAR Framework is [published online](#) for defenders adopting it across their organization.

WHAT TO DO NEXT

Adopting the framework starts with knowing what agents are already operating inside your organization. Most enterprises have more agentic surface than they can name: coding agents in developer IDEs, productivity agents installed by individual employees, first-party agents shipped by internal teams, and MCP servers connected as dependencies, all in production at once. Each inherits whatever controls the organization has applied, or the absence of them.

Discover AI inventories every AI agent, MCP server, and connected model across the enterprise, grading each against the STAR Framework's four layers and three agent types to produce the posture view defenders need.



Discover AI: every agent, model, MCP server, and tool inventoried and graded against the STAR Framework, with per-user outcomes and recommended actions.

Visibility, governance, and posture management are the starting points. The full agentic security stack runs across three control layers: **Discover AI** inventories the estate, **Ascend AI** continuously red-teams the agents Discover AI surfaces with the same adversarial techniques STAR Labs documents in this report, and **Defend AI** runs at runtime to catch the LAVA patterns the model itself cannot recognize.

98.1%	99%	0.7%	<300ms
detection accuracy across threat categories	detection on coding agents at runtime	false-positive rate, near-zero noise	added latency

Defend AI runtime benchmark, measured across the threat categories documented in this report.

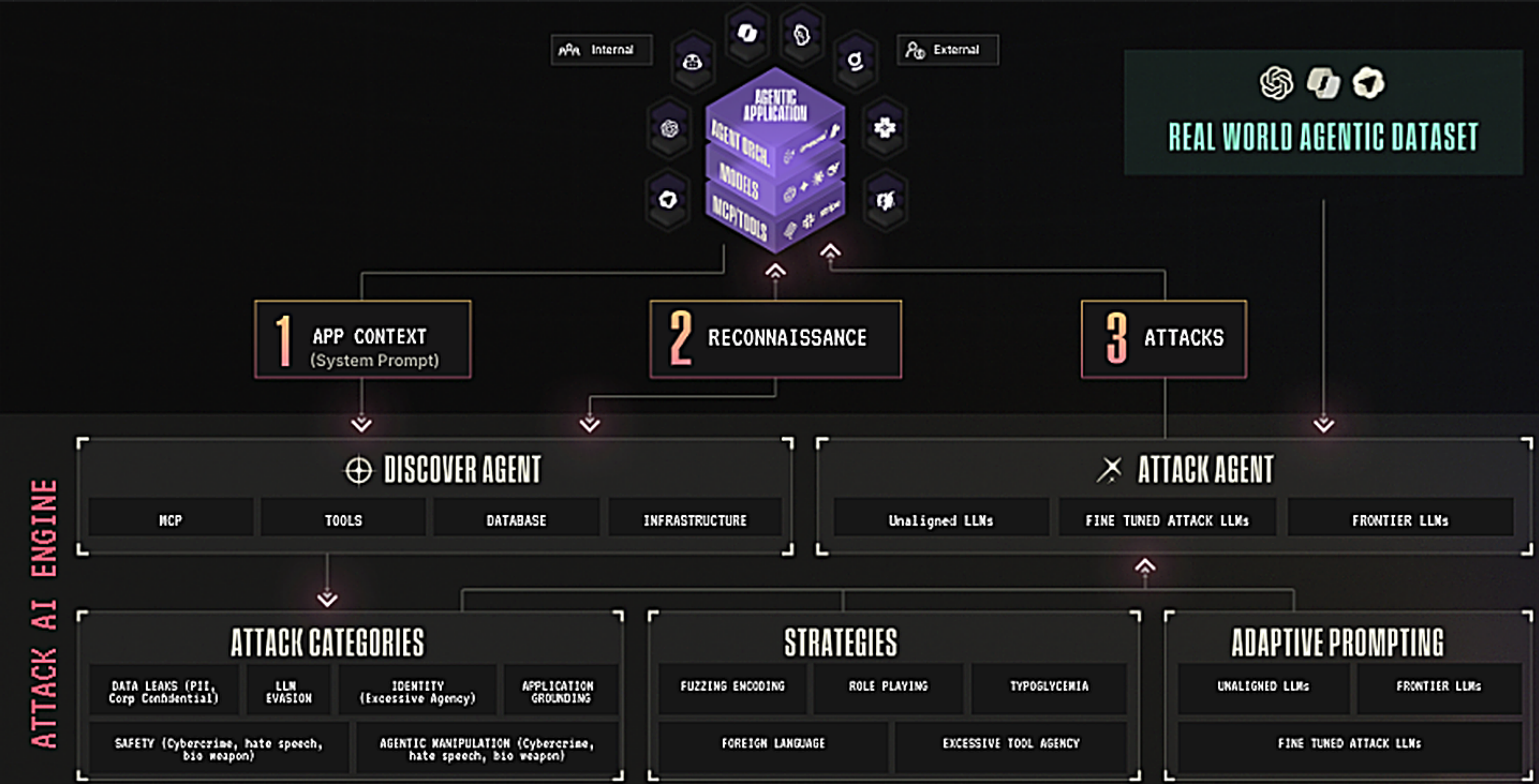
Start with [Discover AI](#). Move to [Ascend AI](#) when you are ready to test the agents you have against the attacks documented here. Add [Defend AI](#) for runtime coverage. Engage at [straiker.ai](#).

METHODOLOGY AND SOURCES

METHODOLOGY

The findings in this report are drawn from STAR Labs' adversarial-engagement corpus and continuous MCP catalog. The engagement corpus represents **1,700+ successful exploits** documented against production AI agents across five form factors (productivity, coding, browser, desktop, CLI) and ten attack categories: data exfiltration, jailbreak, agent harm, tool misuse, indirect prompt injection, agent manipulation, browser harm, remote code execution, tool exploitation, and MCP vulnerabilities. The MCP catalog represents **17,651+ MCP servers** under continuous static, behavioral, and metadata analysis.

Those findings come from **Ascend AI**, Straiker's purpose-built offensive engine, which is vendor-agnostic by design: it does not care which model, framework, or architecture an agent runs. It works in three steps: capture the app context, run reconnaissance, then attack.



The Ascend AI attack engine: app context, reconnaissance, then attack, run by a Discover Agent and an Attack Agent across attack categories, strategies, and adaptive prompting.

Three principles govern the methodology:

- **Real adversarial payloads, not synthetic test cases.** STAR Labs builds the attack infrastructure used in engagements: the malicious websites, weaponized documents, poisoned READMEs, and adversarial MCP servers real attackers would build. Findings reproduce attacker capability rather than approximating it.
- **Severity-rated findings.** Each documented exploit carries a severity classification (critical, high, medium) based on impact, exploitability, and reproducibility. The headline numbers reflect successful-attack rates against deployed agents under realistic conditions.
- **Vendor-agnostic by design.** The engine attacks the context every agent shares, not any one vendor's stack. STAR Labs prioritizes novel patterns that recur across vendors and form factors over deep examination of a single agent, so the findings hold no matter which model, framework, or platform you deploy.

STAY CURRENT

STAR Labs publishes new agentic threat research continuously. Subscribe to the Straiker newsletter on the [blog](#), and bookmark straiker.ai/research for the latest findings.

GLOSSARY

TERM	DEFINITION
AiPT	AI-Powered Persistent Threats: adversaries operating with agentic offensive toolkits. Cyberspike Villager is the canonical example.
LAVA	Language-Augmented Vulnerabilities in Applications: a threat class at the intersection of traditional application vulnerabilities and AI-driven language capabilities.
Straiker STAR Framework	Straiker's framework for AI agent security: four architectural layers across three agent types, with MCP as a shared cross-cutting surface.
Cascade	A multi-step compromise pattern where one agent's compromise propagates across connected systems (Sections 4 and 5).
MCP	Model Context Protocol: a standardized integration protocol for agent tools and resources.

SOURCES

STAR LABS RESEARCH

- Nomshub: Cursor Remote Tunneling Sandbox Breakout
- Claude Code Source Leak: With Great Agency Comes Great Responsibility
- Built on ClawHub, Spread on Moltbook: The New Agent-to-Agent Attack Chain
- From Inbox to Wipeout: An AI Browser Quietly Erasing a Cloud Drive
- Fake Claude Code, Real Malware: Inside the Campaign Targeting AI Developers
- Cyberspike Villager: AI-Native Cobalt Strike Lookalike

PUBLIC REPORTING AND INCIDENTS

- Hackers hijacked Instagram accounts by tricking Meta's AI support chatbot (TechCrunch)
- Cursor-Opus agent snuffs out startup's production database (The Register)
- Google Unveils Gemini Enterprise Agent Platform (Alwire)
- Snowflake Cortex Agents: Enterprise AI Agent Platform (Snowflake)
- Amazon Bedrock AgentCore general availability (AWS)

FRAMEWORK REFERENCES

- OWASP Top 10 for LLM Applications
- OWASP Top 10 for Agentic Applications 2026
- Google Secure AI Framework (SAIF)
- MITRE ATLAS
- NIST AI Risk Management Framework (AI 100-1)

FIGURES & TABLES

The STAR Framework: four architectural layers across three...	5	Chart: MCP vulnerabilities by severity	26
One attack, four layers: a single compromise enters at Data...	7	Chart: High-risk tool exposure by category	26
The three deployment roles, by trust posture, primary attac...	8	Table: critical and high findings by runtime stack	26
Four generations, one rising risk curve. Generation 3 is whe...	10	The STAR Framework as a control map: four cross-cutting...	28
From a poisoned README to full compromise, and the contr...	12	Discover AI: every agent, model, MCP server, and tool inve...	29
Products cloned: Claude Code . JetBrains . NotebookLM . C...	14	Defend AI runtime benchmark, measured across the threat...	29
Hosted across: Squarespace . GitHub Pages . Cloudflare W...	14	The Ascend AI attack engine: app context, reconnaissance...	30
The blast radius: one compromise, three radii	22		